Nombre: Jose Gerardo Ortiz Leñero	**Matrícula**: AL07182099
Nombre del curso: Gestión de proyectos de inteligencia artificial **Grupo:** CC.MSCA2102FLE.102.202630	**Nombre del profesor**: Ing. Tania Judith Mendoza Torres
Módulos: 11. Gestión de datos para entrenamiento. 12. Entrenamiento y ajuste de modelos.	**Actividad**: Act. 5. Preparación de datos, entrenamiento y registro de modelos con MLflow.
Fecha: 18/Oct/2026	
MASTER INTELIGENCIA ARTIFICIAL	

Universidad Tecmilenio

Tabla de Contenido

Introducción **3**

1.- Descripción del dataset **3**

2.- Proceso de limpieza y estandarización **4**

3.- Estrategia de versionamiento **5**

4.- Patrones y anomalías (visualizaciones). **6**

5.- Modelos entrenados **8**

6.- Configuración de hiperparámetros **9**

7.- Métricas obtenidas **10**

8.- Comparación estadística de resultados **11**

9.- Evidencia del uso de MLflow **13**

10.- Deuda técnica y MLOps **17**

11.- Conclusión **18**

Bibliografía ***19***

Introducción

Dando continuidad al proyecto de DEMANDA-IA, donde propone que Farmacias Occidente Salud genere cada noche un pronóstico de demanda por SKU y sucursal que alimente el reabasto del ERP. En esta actividad se construye el primer baseline del componente central de la Fase I. Primero se prepara y versiona un dataset temporal SKU–sucursal (Universidad Tecmilenio., s.f.-a). Después se entrenan y ajustan tres algoritmos frente a una referencia ingenua, y todo el proceso se registra en MLflow (Universidad Tecmilenio., s.f.-b) para que cualquier resultado pueda rastrearse hasta el código, los datos y los hiperparámetros que lo produjeron.

El trabajo se entrega como un repositorio Git (Actividad5/). Contiene los datos originales y limpios, el módulo de limpieza datos_prep.py, el script de entrenamiento train.py, el cuaderno entrena.ipynb, el backend de MLflow (mlflow.db y mlruns/), la evidencia visual, el README.md y el CHANGELOG.md. Cada sección de este reporte corresponde a un inciso (a–j) de las instrucciones.

1.- Descripción del dataset

Fuente

Se utilizó el conjunto público Store Item Demand Forecasting Challenge (Kaggle, 2018), archivo train.csv. Contiene las ventas diarias de 50 artículos en 10 tiendas entre el 1/01/2013 y el 31/12 /2017. 913,000 registros que forman 500 series temporales completas. La copia descargada tiene la huella SHA-256 038f2569…92f2ed, registrada en datos/MANIFIESTO_DATOS.json.

Justificación. El extracto real del ERP de la cadena aún no está disponible. Este dataset se eligió como proxy estructural porque tiene exactamente la granularidad que requiere DEMANDA-IA (fecha × sucursal × SKU) y la misma variable objetivo (unidades diarias). Así, el pipeline, las métricas y la infraestructura de MLflow quedan listos para recibir los datos reales sin cambios de diseño.

Diccionario de datos

Columna original	Columna limpia	Tipo	Descripción
date	fecha	fecha	Día de la venta (AAAA-MM-DD)
store	sucursal_id / codigo_sucursal	int16 / texto	Sucursal 1–10 → SUC-01 … SUC-10
item	sku_id / codigo_sku	int16 / texto	Artículo 1–50 → SKU-001 … SKU-050
sales	unidades	int32	Unidades vendidas en el día (variable objetivo)
—	es_imputado	0/1	Valor imputado por la regla R7
—	es_devolucion	0/1	Valor original negativo, acotado a 0 (R5)
—	es_anomalia	0/1	|z robusto| > 3.5 (R8); se conserva
—	z_robusto	float32	Desviación robusta frente a la mediana móvil centrada de 28 días

Tabla 1. Diccionario de datos del dataset limpio (v1.1.0)

Supuestos

- «Tienda» equivale a sucursal y «artículo» equivale a SKU.

- Las ventas se interpretan como unidades dispensadas. No se distingue la demanda censurada por desabasto, porque no hay datos de inventario.
- No hay precios, promociones, calendario mexicano ni variables externas. Se incorporarán con los datos reales (Sección 10).
- Los perfiles de SKU son casi idénticos entre sucursales (correlación media de 0.9999), lo que indica que el dataset es muy regular y probablemente sintético. Por eso, los errores absolutos obtenidos son una cota optimista del desempeño con datos reales de farmacia.

2.- Proceso de limpieza y estandarización

El módulo fuentes/datos_prep.py aplica nueve reglas en orden fijo. Cada regla tiene un criterio explícito y su efecto queda contabilizado en datos/datos_limp/reporte_calidad.json, de modo que el proceso es auditable y reproducible.

Regla	Criterio	Resultado
R1 Esquema	Renombrar a snake_case en español; forzar tipos (fecha=datetime, ids=int, unidades=float)	4 columnas tipadas. 0 nulos iniciales
R2 Llave inválida	Eliminar fechas no parseables o ids nulos o no positivos (no se pueden atribuir a una serie)	0 eliminadas
R3 Duplicados exactos	Eliminar filas idénticas (doble carga del archivo POS)	0
R4 Duplicados de llave	Ante distinta venta para la misma fecha-sucursal-SKU, conservar el último registro cargado	0
R5 Negativos	Las devoluciones no son demanda: acotar a 0 y marcar es_devolucion	0
R6 Calendario	Completar la rejilla diaria por serie; un día faltante es dato faltante, no venta cero	0 días agregados
R7 Imputación	Mediana de la misma serie y día de la semana en las 4 semanas previas (respaldo: mediana de la serie)	0 imputados
R8 Anomalías	|z robusto| > 3.5 (Iglewicz y Hoaglin, 1993) contra la mediana móvil centrada de 28 días; marcar, no eliminar	1,412 (0.15 %)
R9 Códigos	Generar SUC-XX y SKU-XXX para la integración con el ERP	500 series codificadas

Tabla 2. Reglas de limpieza, criterio y resultado sobre el dataset.

El dataset público no traía nulos, duplicados ni negativos. Para demostrar que las reglas sí actúan ante una exportación real de punto de venta, pruebas/test_datos_prep.py contamina deliberadamente una muestra. Inyecta 2 llaves inválidas, 2 duplicados exactos, 1 conflicto de llave, 1 nulo, 1 devolución, 2 días faltantes y 1 pico. La prueba verifica que cada regla corrija exactamente lo esperado. R7 imputa 3 valores, que corresponden al nulo y a los dos días faltantes.

Escalado y codificación. Las transformaciones de Ridge se definen dentro de su pipeline para que no haya fuga de información entre folds. Ahí, las categóricas pasan por OneHotEncoder y las numéricas por StandardScaler. El escalado es necesario porque la penalización L2 castiga a todos los coeficientes por igual, y si las variables están en escalas distintas, unas terminan más penalizadas que otras.

Random Forest y XGBoost reciben las numéricas sin escalar, porque las particiones de un árbol no cambian ante transformaciones monótonas, y las categóricas como enteros, es decir, con codificación ordinal.

En el día de la semana y el mes eso funciona bien porque tienen un orden real que los cortes aprovechan. En sucursal y SKU el orden es arbitrario, pero con solo 10 y 50 niveles el árbol puede aislar cualquier valor con un par de cortes.

Ingeniería de variables. Se generan 17 variables:

- Rezagos de 7, 14, 21, 28 y 364 días.
- Media móvil de 7 y de 28 días, y desviación estándar de 28 días, calculadas a partir de t−7.
- media_dow_4: promedio del mismo día de la semana en las 4 semanas previas.
- Calendario (día de la semana, mes, día del mes, semana del año, fin de semana), tendencia, sucursal y SKU.

Todas usan información con al menos 7 días de antigüedad. Esta es una regla de negocio: el pronóstico nocturno debe cubrir el tiempo de reabasto. También garantiza que no haya fuga de información.

3.- Estrategia de versionamiento

Elemento	Mecanismo	Identificador
Datos originales	Etiqueta Git datos-v1.0.0 + SHA-256 en el manifiesto	038f2569…92f2ed
Datos limpios	Etiqueta Git datos-v1.1.0 + SHA-256 del contenido lógico	fb5fa9b3…5d51c
Código de entrenamiento	Commit Git registrado como tag git_commit en cada corrida	8122fc5
Baseline entrenado	Etiqueta Git modelo-v1.2.1	7ad2b6b
Modelos	Registro de MLflow demanda_ia_pronostico_sku_sucursal	v1 ridge, v2 RF, v3 XGBoost (@campeon)
Entorno	Versiones registradas como etiquetas en cada corrida de MLflow	Python 3.13, pandas 2.2.3, numpy 2.1.3, sklearn 1.6.1, xgboost 3.4.1, mlflow 3.17.1

Tabla 3. Componentes versionados y su identificador.

Los cambios se documentan en CHANGELOG.md con el formato *Keep a Changelog* y versionado semántico. Cada corrida de MLflow guarda el SHA-256 del dataset y el commit de código. Con esos dos datos, cualquier métrica puede rastrearse hasta la versión exacta de los datos y del código que la produjo. El hash del contenido se calcula sobre el DataFrame y no sobre el archivo, para que no cambie si se recomprime el archivo.

Repositorio en GitHub

Todo el proyecto vive en un repositorio Git publicado en GitHub (https://github.com/Gerardortizle/Actividad5). Se creó con la estructura mínima solicitada y se agregaron las carpetas para la trazabilidad y la evidencia:

```
Actividad5/
├── datos/
│   ├── datos_ini/train.csv                                  # datos originales (v1.0.0)
│   ├── datos_limp/ventas_sku_sucursal_limpio.csv.gz         # datos limpios (v1.1.0)
│   ├── datos_limp/reporte_calidad.json                      # conteo por regla R1–R9
│   └── MANIFIESTO_DATOS.json                                # versión + SHA-256
├── fuentes/
│   ├── entrena.ipynb                # cuaderno de Colab: EDA, entrenamiento y evaluación
│   ├── datos_prep.py                # funciones de limpieza e ingeniería de variables
│   └── train.py                     # Grid Search + validación temporal + MLflow
├── pruebas/test_datos_prep.py       # prueba de robustez de la limpieza
├── resultados/                      # métricas, comparación estadística, predicciones
├── evidencias/graficas/             # figuras del EDA y de resultados
├── evidencias/mlflow/               # capturas del panel de MLflow
├── mlflow.db                        # backend de seguimiento (SQLite)
├── mlruns/                          # artefactos de MLflow
├── reporte/                         # este reporte (Word y PDF)
├── requirements.txt                 # versiones exactas
├── README.md                        # guía de reproducción y decisiones
└── CHANGELOG.md                     # historial de versiones
```

El README.md documenta cómo reproducir todo (en Colab y en local), las decisiones técnicas y el diccionario de datos. El .gitignore excluye cachés de Python y el binario del Random Forest (130.8 MB), que supera el límite de 100 MB por archivo de GitHub (Sección 10).

4.- Patrones y anomalías (visualizaciones).

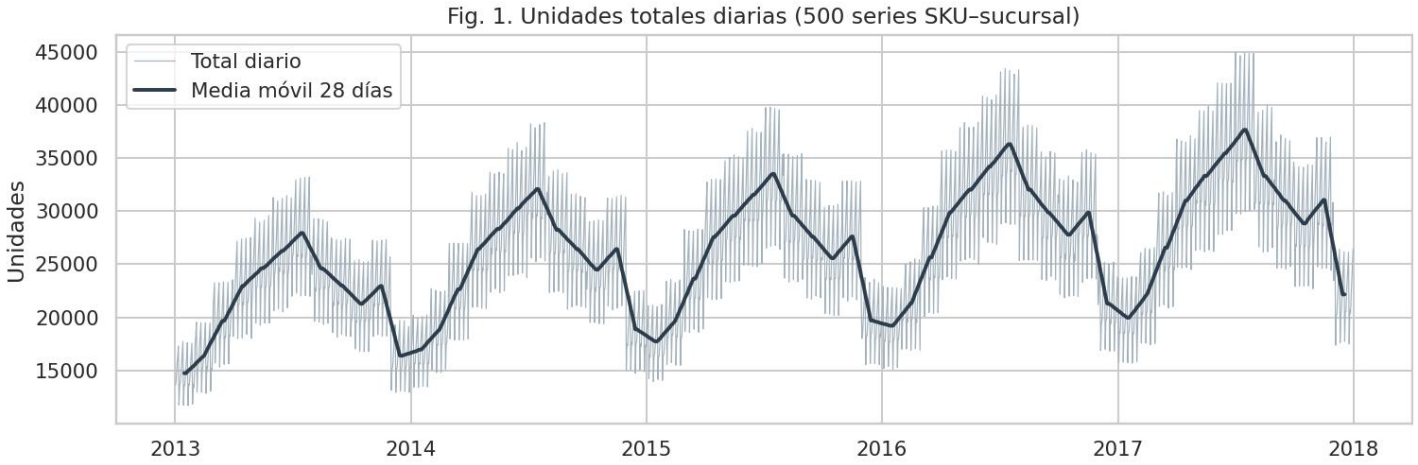


Figura 1. Unidades totales diarias y media móvil de 28 días.

Tendencia y estacionalidad anual. La demanda total crece cada año, +15.0 % en 2014, +4.4 % en 2015, +8.6 % en 2016 y +3.6 % en 2017. El ciclo anual se repite con un pico en julio y un valle en enero. Esto justifica dos decisiones, incluir el rezago de 364 días y una variable de tendencia, y validar con bloques que abarquen ambas mitades del año.

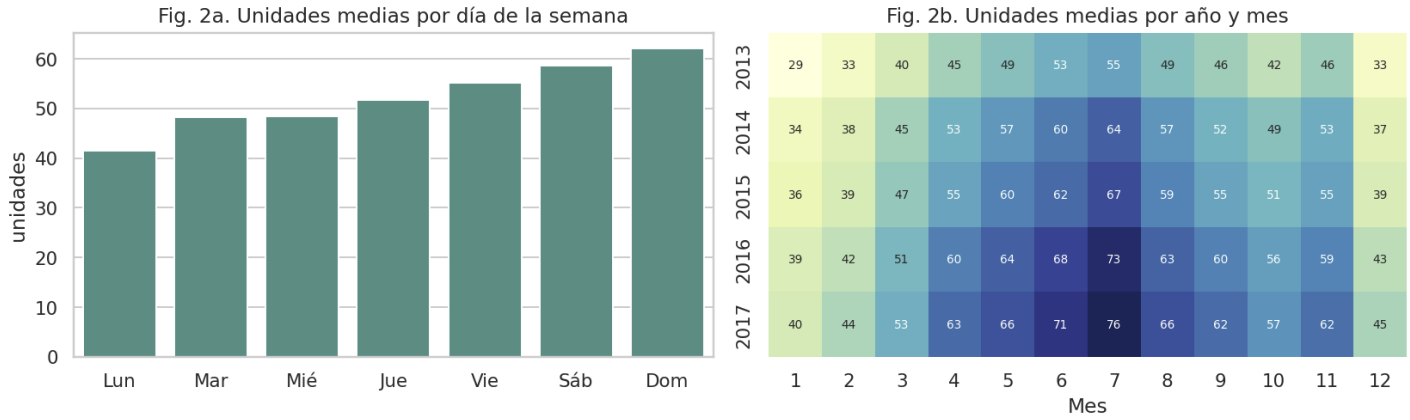


Figura 2. Estacionalidad semanal (a) y mensual por año (b).

Estacionalidad semanal. Con respecto a la media, el lunes vende 21 % menos y el domingo 19 % más, con un aumento monótono a lo largo de la semana. Por eso, la referencia naive usa el mismo día de la semana anterior (lag 7) y no el día anterior (Hyndman, R. J., & Athanasopoulos, G., 2021).

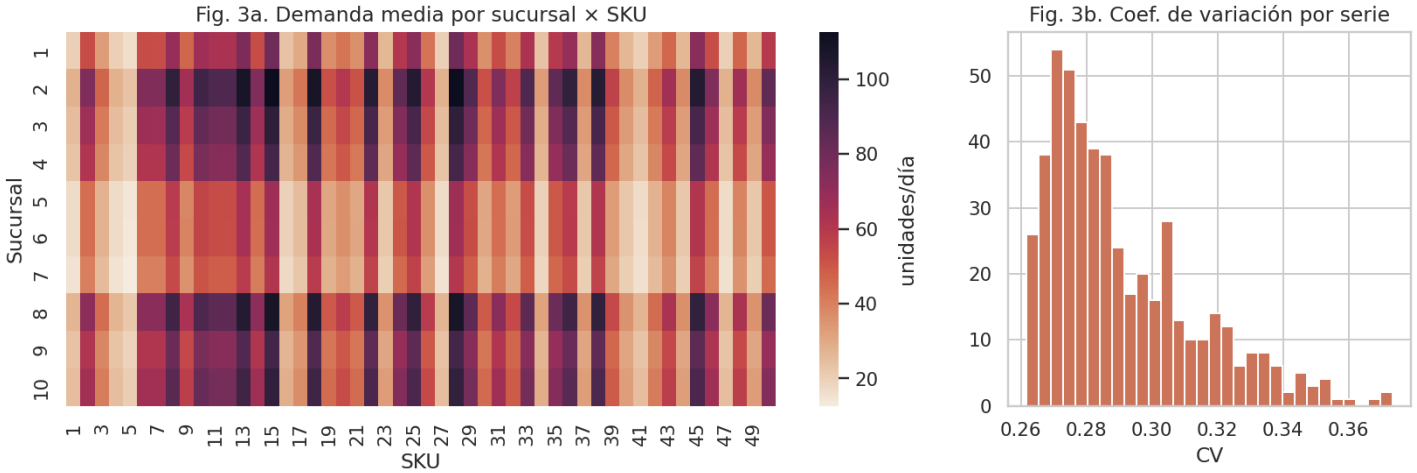


Figura 3. Demanda media por sucursal x SKU (a) y coeficiente de variación por serie (b).

Heterogeneidad. La demanda media por serie va de 12.7 a 112.6 unidades diarias. Esa diferencia de casi 9× entre series motiva el uso del WAPE como métrica de negocio, porque es comparable entre SKU de distinto volumen. El coeficiente de variación es bajo y uniforme (0.26–0.37), algo atípico en farmacia, donde muchos SKU tienen demanda intermitente.

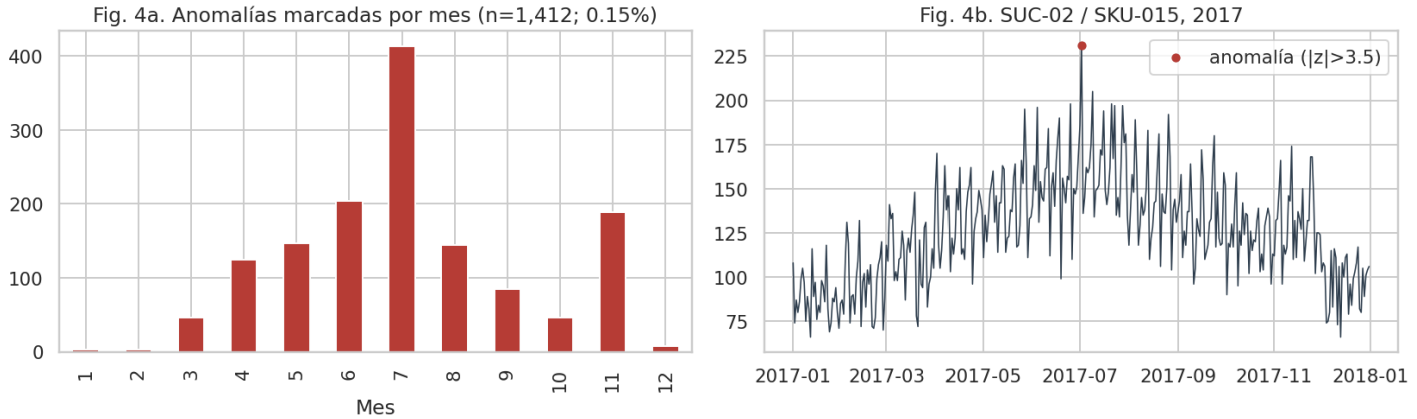


Figura 4. Anomalías por mes (a) y ejemplo en la serie SUC-02/SKU-015 (b).

Anomalías. Se marcaron 1,412 observaciones (0.15 %): 1,286 picos y 126 caídas. Se concentran en julio y noviembre, los meses de cambio estacional más brusco, donde la mediana móvil se queda atrás. Esto confirma la decisión de marcar y no eliminar: casi todas son picos estacionales legítimos. En una farmacia, el equivalente sería un brote respiratorio, que es justo la señal que el modelo debe aprender.

Fig. 5. Autocorrelación del total diario (líneas: rezagos 7, 14, 28 y 364)

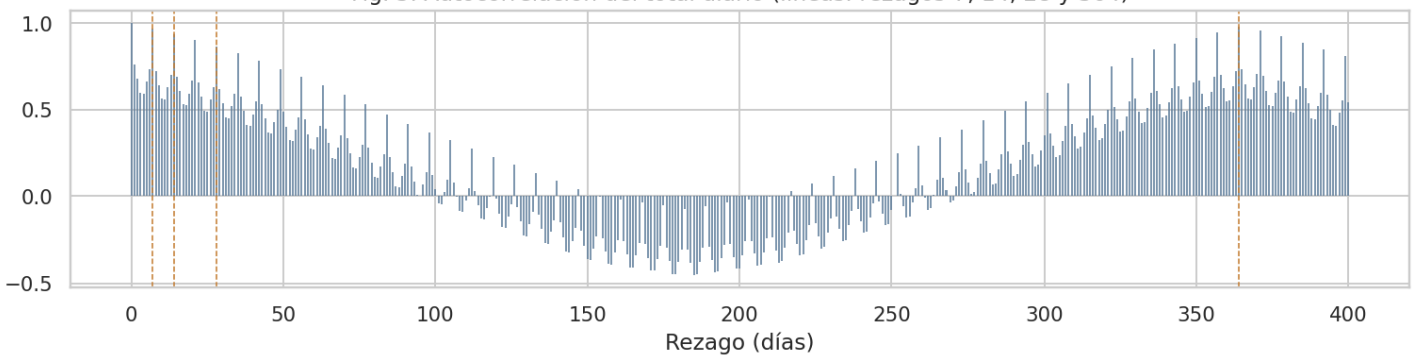


Figura 5. Autocorrelación del total diario.

Dependencia temporal. La autocorrelación es de 0.966 en el rezago 7, 0.933 en el 14 y 0.865 en el 28, y alcanza 0.980 en el rezago 364. En cambio, es de −0.074 a mitad de año (rezago 182). Estos valores respaldan empíricamente la selección de rezagos.

Fig. 6. Esquema temporal: ventana expansiva (3 folds de 6 meses) + holdout jul–dic 2017

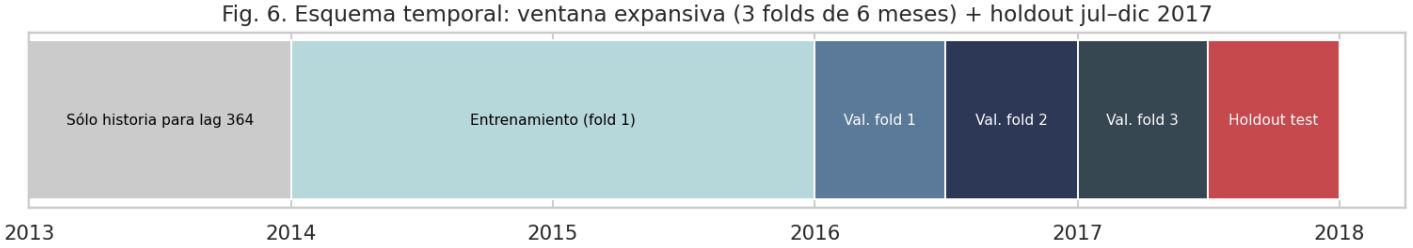


Figura 6. Esquema de validación temporal.

5.- Modelos entrenados

Modelo	Tipo	Rol y justificación
Naive estacional	ŷ(t) = y(t−7)	Referencia mínima. Un modelo que no la supere no aporta valor.
Ridge	Lineal con penalización L2	Modelo interpretable y barato; la penalización controla la multicolinealidad de rezagos y medias móviles (Hoerl, A. E., & Kennard, R. W., 1970).
Random Forest	Ensamble por bagging	Captura no linealidades e interacciones sucursal × SKU × calendario. Robusto a valores atípicos (Breiman, L., 2001).
XGBoost	Ensamble por gradient boosting	Suele ser el estado del arte en datos tabulares, corrige secuencialmente los residuos y admite pérdidas para conteos (Chen, T., & Guestrin, C. , 2016).

Tabla 4. Algoritmos comparados.

Los tres son modelos globales. Una sola instancia aprende las 500 series a la vez, usando sucursal y SKU como variables. Para Farmacias Occidente Salud, con 140 sucursales, esto es crucial, un modelo por serie implicaría decenas de miles de modelos que mantener.

6.- Configuración de hiperparámetros

Se usó GridSearchCV de scikit-learn (Pedregosa, F., et al., 2011) con validación cruzada temporal de ventana expansiva en 3 folds. Cada fold entrena con toda la historia previa y valida un bloque de 6 meses: 2016-S1, 2016-S2 y 2017-S1. Un k-fold aleatorio no es válido en series de tiempo, porque usaría el futuro para predecir el pasado (Bergmeir, C., & Benítez, J. M., 2012). El periodo jul–dic 2017 (184 días, 92,000 observaciones) queda como holdout intocable.

Por costo computacional, la búsqueda se hizo sobre una muestra aleatoria del 12 % del entrenamiento (76,680 filas). Después, cada modelo se reentrenó con el 100 % (639,000 filas) usando los mejores hiperparámetros. La métrica de selección fue el MAE.

Modelo	Rejilla	Justificación técnica	Mejor (MAE CV)
Ridge	alpha ∈ {0.01, 0.1, 1, 10, 100}	Escala logarítmica de regularización L2	α=100 (6.463)
Random Forest	max_depth ∈ {10, 16}; min_samples_leaf ∈ {10, 30}; max_features ∈ {0.5, 1.0}. Fijos: 100 árboles, max_samples=0.3	Profundidad y tamaño de hoja controlan el sobreajuste; max_features decorrelaciona los árboles; max_samples acota el costo	16 / 10 / 0.5 (6.445)
XGBoost	max_depth ∈ {4, 6, 8}; learning_rate ∈ {0.05, 0.1}; objective ∈ {squarederror, count:poisson}. Fijos: 500 árboles, subsample=0.8, colsample=0.8	Complejidad de cada árbol frente a tasa de aprendizaje; la pérdida Poisson es la natural para conteos de unidades	6 / 0.05 / squarederror (6.185)

Tabla 5. Rejillas de búsqueda (25 combinaciones x 3 folds = 75 ajustes).

Lectura de la búsqueda

- **Ridge.** Es prácticamente insensible a α: el MAE CV va de 6.463 a 6.467. Con 639 mil filas y unas cien variables, la regularización casi no actúa. Esto indica que el límite del modelo es su linealidad, no el sobreajuste.
- **Random Forest.** Prefiere hojas pequeñas (10 frente a 30) y, con ellas, max_features=0.5 iguala o supera a 1.0 con la mitad del tiempo de ajuste.
- **XGBoost.** La mejor combinación es la de profundidad media con tasa de aprendizaje baja. La pérdida Poisson queda en segundo lugar a solo 0.012 unidades. Con volúmenes medios de ~52 unidades diarias, la distribución de conteos se aproxima a la normal y ambas pérdidas convergen. Se espera que Poisson (o Tweedie) gane con los SKU de baja rotación de la farmacia real.

Entorno y documento de ejecución

El entrenamiento se ejecuta desde el cuaderno fuentes/entrena.ipynb, diseñado para Google Colab y también ejecutable en local. El cuaderno no duplica la lógica de entrenamiento, la sección 5 invoca el script versionado fuentes/train.py, de modo que el resultado es idéntico al de la línea de comandos y queda ligado a un commit de Git.

Las secciones del cuaderno son:
- 1. Entorno: en Colab clona el repositorio e instala MLflow y XGBoost
- 2–3. Datos y limpieza: carga datos_ini, muestra el diccionario, ejecuta datos_prep.main() y la prueba de robustez.

- 4. Análisis exploratorio: genera las Figuras 1–6 en evidencias/graficas/.

- 5. Entrenamiento: ejecuta python fuentes/train.py (Grid Search, reentrenamiento, evaluación y registro en MLflow).
- 6. Resultados: tablas de métricas, Figuras 7–10 y pruebas estadísticas.
- 7. Trazabilidad: consulta las corridas con mlflow.search_runs() y carga el modelo campeón desde el registro.

La corrida reportada se ejecutó en Google Colab con Python 3.13, pandas 2.2.3, numpy 2.1.3, scikit-learn 1.6.1, XGBoost 3.4.1 y MLflow 3.17.1; el proceso completo de train.py tardó 405 s. Las versiones de cada corrida quedan registradas como etiquetas en MLflow..

7.- Métricas obtenidas

Modelo	MAE	RMSE	R^2	WAPE	Sesgo	MAE CV (media ± d.e.)	Entren. (s)
Naive estacional	9.214	12.114	0.859	15.0 %	+1.63 %	8.600 ± 0.317	0
Ridge	6.868	8.871	0.924	11.2 %	+0.31 %	6.463 ± 0.245	1.5
Random Forest	6.521	8.462	0.931	10.6 %	+0.64 %	6.445 ± 0.216	71.9
XGBoost	6.271	8.126	0.936	10.2 %	+0.31 %	6.185 ± 0.174	20.0

Tabla 6. Métricas en el holdout jul-dic 2017 (n = 92,000) y en la validación cruzada.

El problema es de regresión, así que se reportan las métricas indicadas: MAE, RMSE y R^2. Se añaden el WAPE, que es la suma de errores absolutos entre la suma de la demanda y es el estándar en retail, y el sesgo porcentual, que es crítico para el inventario. Un sesgo positivo genera sobreinventario; uno negativo, desabasto.

Interpretación:

- XGBoost comete en promedio un error de 6.27 unidades por SKU–sucursal–día, equivalente a 10.2 % del volumen. Esto representa un 32 % menos que la referencia naive.

- Todos los modelos tienen un sesgo menor a 1 %, por lo que no sobreestiman ni subestiman sistemáticamente.

- El MAE CV es menor en los folds S1 que en 2016-S2 para todos los modelos (Fig. 7a). Esto no es inestabilidad: el MAE crece con el volumen y el segundo semestre incluye el pico de julio. La correlación entre volumen medio y MAE por serie es de 0.97, mientras que la del WAPE es de −0.90. Es decir, las series de bajo volumen son relativamente más difíciles; su WAPE va de 6.9 % a 21.8 %.

- Ridge muestra la mayor brecha entre CV (6.46) y holdout (6.87). Un modelo lineal extrapola peor la interacción entre tendencia y estacionalidad en el semestre de mayor volumen.

8.- Comparación estadística de resultados

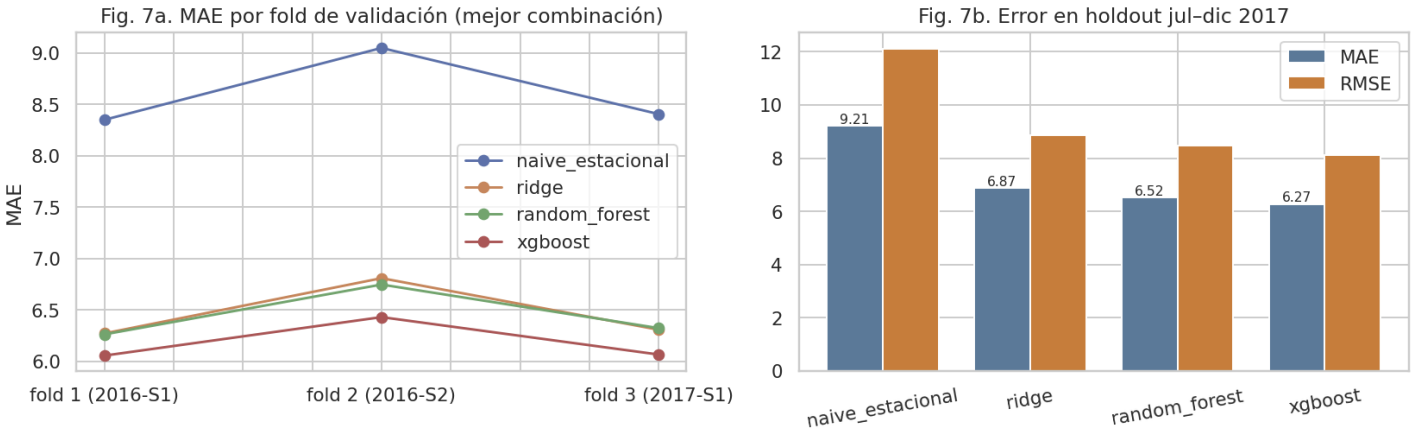


Figura 7. MAE por fold de validación (a) y error en el holdout (b).

La unidad de comparación es el MAE por serie (n = 500 pares). Este enfoque es más exigente que comparar un único número agregado, porque verifica que la mejora ocurra de forma consistente en las series.

- **Friedman** (no paramétrica, k modelos relacionados; Demšar, 2006): $\chi^2 = 1410.4$, $p < 10^{-300}$. Se rechaza la hipótesis de que los cuatro modelos sean equivalentes. Rangos promedio: XGBoost 1.05, Random Forest 2.05, Ridge 2.91 y naive 3.99.

- **Comparaciones por pares:** Wilcoxon de rangos con signo, con corrección de Holm por comparaciones múltiples, y t pareada como contraste paramétrico.

- **Diebold–Mariano** con la corrección de Harvey et al. (1997) y varianza HAC con 6 rezagos, sobre la pérdida absoluta diaria. Contempla la autocorrelación de los errores de pronóstico.

- **Intervalo bootstrap** al 95 % de la diferencia de MAE (2,000 remuestreos de series).

Comparación (A vs B)	Δ MAE (A−B)	IC 95%	Series donde gana A	Wilcoxon p (Holm)	d de Cohen	DM	DM p
Naive vs Ridge	2.346	[2.27, 2.42]	5 / 500	< 0.001	2.57	7.77	< 0.001
Naive vs RF	2.693	[2.62, 2.77]	0 / 500	< 0.001	3.09	6.95	< 0.001
Naive vs XGBoost	2.944	[2.86, 3.03]	0 / 500	< 0.001	2.98	7.32	< 0.001
Ridge vs RF	0.347	[0.32, 0.37]	45 / 500	< 0.001	1.12	2.44	0.016
Ridge vs XGBoost	0.598	[0.56, 0.63]	5 / 500	< 0.001	1.69	3.87	< 0.001
RF vs XGBoost	0.250	[0.23, 0.27]	18 / 500	< 0.001	1.38	9.23	< 0.001

Tabla 7. Pruebas por pares. Un Δ positivo indica que B es mejor.

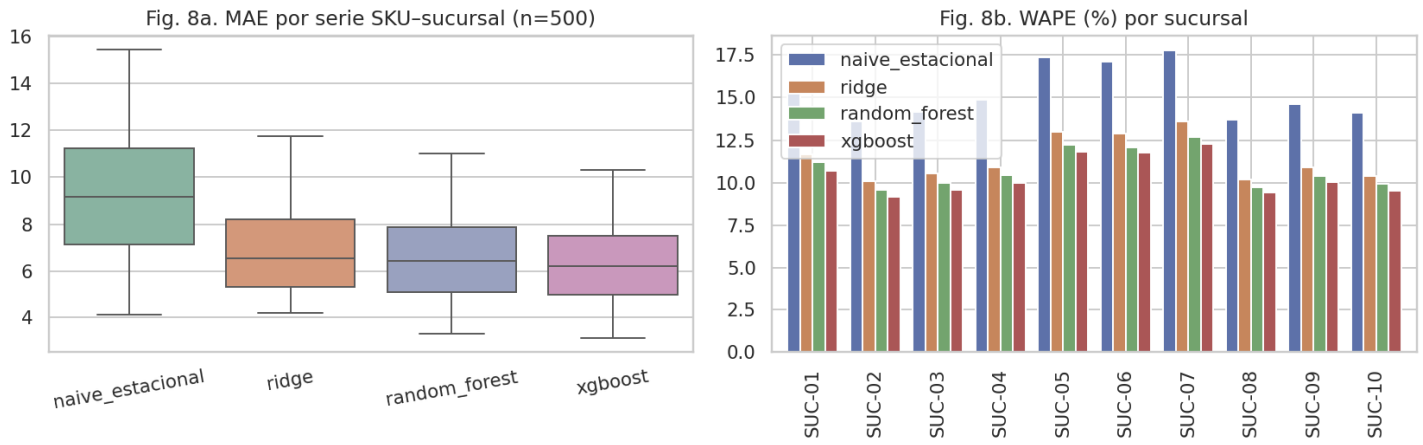


Figura 8. Distribución del MAE por serie (a) y WAPE por sucursal (b).

Conclusión estadística. XGBoost supera a Random Forest en 482 de 500 series. La diferencia media es de 0.25 unidades, con un IC al 95 % de [0.23, 0.27] que no incluye el cero, y un efecto grande (d = 1.38). Diebold–Mariano lo confirma en el horizonte temporal ($p < 0.001$).

La diferencia entre Ridge y Random Forest es la menos contundente: DM da $p = 0.016$ y el modelo lineal gana en 45 series. En la Figura 8b, el ordenamiento de los modelos se mantiene en las 10 sucursales. El WAPE de XGBoost va de 9.2 % (SUC-02, la de mayor volumen) a 12.3 % (SUC-07, la de menor volumen), lo que confirma que el volumen es el principal determinante del error relativo.

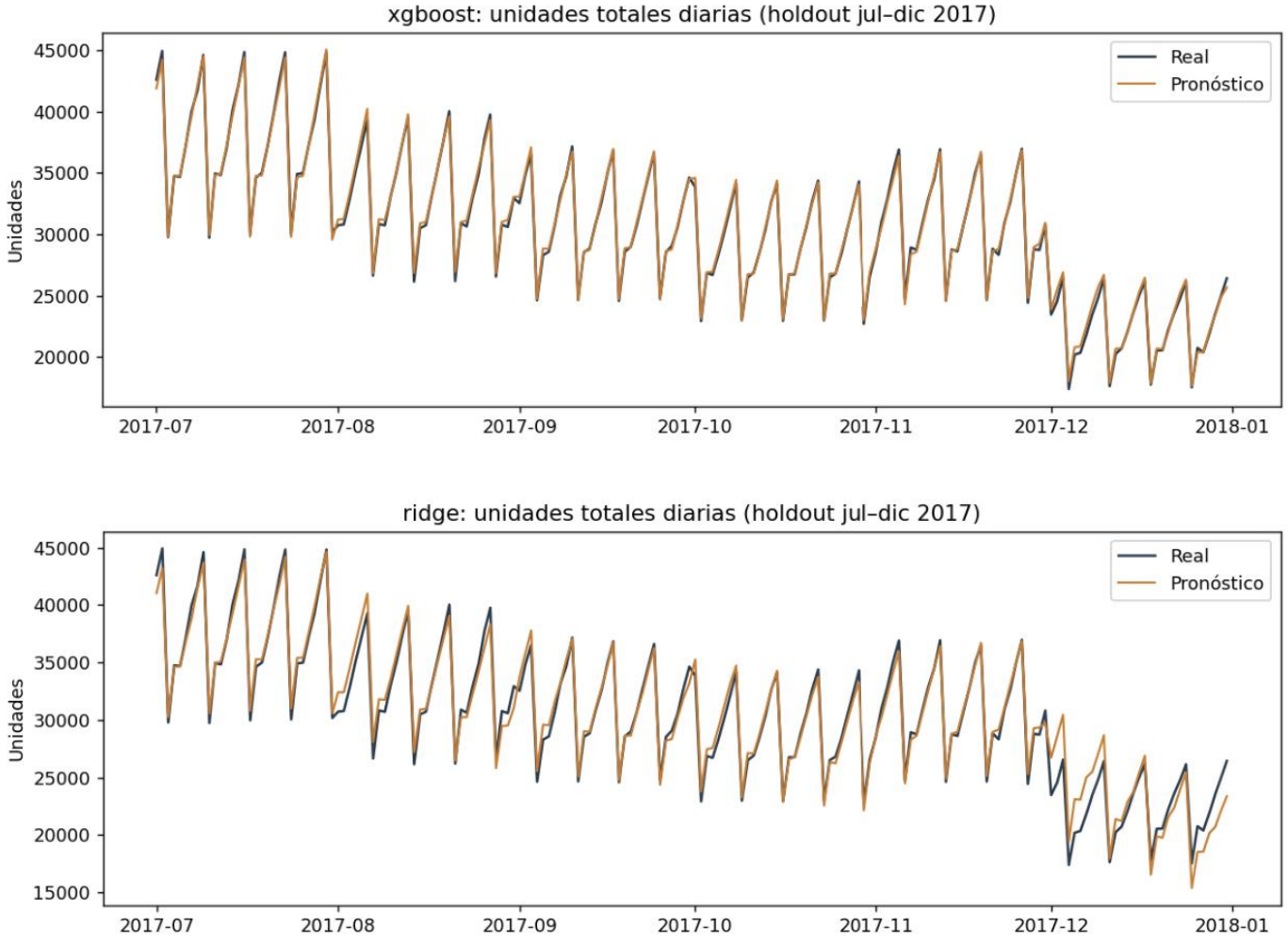


Figura 9. Real frente a pronóstico del total diario en el holdout. XGBoost (arriba) y Ridge (abajo).

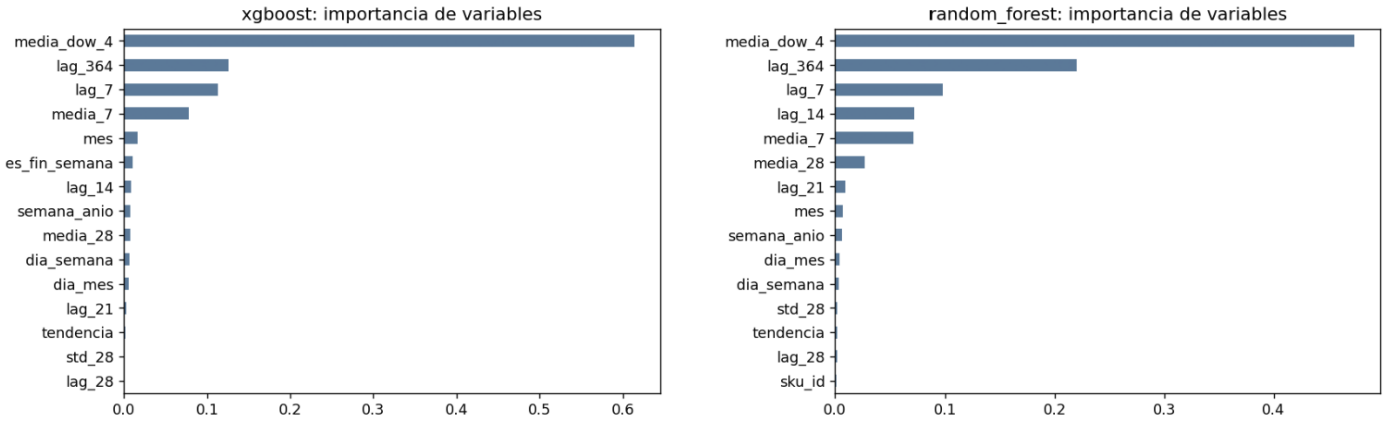


Figura 10. Importancia de variables en XGBoost y Random Forest.

Interpretación de las variables. En ambos ensambles domina media_dow_4, el promedio del mismo día de la semana en las 4 semanas previas, seguida del rezago de 364 días. Los modelos aprenden un nivel reciente ajustado por día de la semana, corregido por la estacionalidad anual, que es justo la estructura que mostraron las Figuras 2 y 5. Las variables de identidad (sucursal y SKU) pesan poco porque el nivel de cada serie ya está contenido en sus rezagos.

9.- Evidencia del uso de MLflow

Configuración

Elemento	Configuración
Modo de seguimiento	Local, dentro del repositorio (portátil a Colab y a cualquier equipo)
Backend de metadatos	SQLite: sqlite:///mlflow.db (corridas, parámetros, métricas, etiquetas y registro de modelos)
Almacén de artefactos	Carpeta ./mlruns (gráficas, CSV, modelos serializados)
Experimento	DEMANDA-IA_FaseI_baseline_SKU_sucursal, con descripción y etiquetas proyecto, fase, cliente y tarea
Nomenclatura de corridas	00_naive…, 01_ridge, 02_random_forest, 03_xgboost, 99_comparacion; las hijas se nombran modelo|hiperparámetros
Registro de modelos	demanda_ia_pronostico_sku_sucursal, versiones 1–3, alias campeon → v3
Visualización	mlflow ui --backend-store-uri sqlite:///mlflow.db → http://127.0.0.1:5000

Tabla 8. Configuración de MLflow.

Fragmento de train.py que establece el seguimiento y la jerarquía de corridas:

```
mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("DEMANDA-IA_FaseI_baseline_SKU_sucursal")
with mlflow.start_run(run_name="03_xgboost"):                          # corrida padre
    mlflow.set_tags({"git_commit": ..., "dataset_sha256": ...})
    mlflow.log_input(dataset_ml, context="entrenamiento")
    for combinacion in grid:                                    # 12 corridas hijas
        with mlflow.start_run(run_name="xgboost|...", nested=True):
            mlflow.log_params(...); mlflow.log_metrics({"cv_mae_media": ...})
    mlflow.log_metric("cv_mae_fold", mae, step=fold)
    mlflow.log_metrics({"test_mae": ..., "test_rmse": ..., "test_r2": ...})
    mlflow.xgboost.log_model(modelo, name="modelo", signature=firma,
                             registered_model_name="demanda_ia_pronostico_sku_sucursal")
```

En Colab, mlflow.db y mlruns/ se crean dentro del repositorio clonado. Para revisar el panel se descargan ambos al equipo local y se abre mlflow ui desde la raíz del repositorio; las capturas de esta sección se tomaron así.

Contenido registrado

El seguimiento usa un backend SQLite (mlflow.db) con artefactos en mlruns/ (MLflow Project., 2026). El experimento DEMANDA-IA_FaseI_baseline_SKU_sucursal tiene una descripción y las etiquetas de proyecto, fase y cliente. Sus 30 corridas se organizan así:

- 1 corrida de referencia (naive).
- 3 corridas padre, una por algoritmo.
- 25 corridas hijas anidadas, una por combinación del grid, con sus parámetros y el MAE, RMSE y R^2 medios de la validación cruzada.
- 1 corrida con la comparación estadística.

Cada corrida padre registra:

- Parámetros comunes (semilla, horizonte, cortes, fracción de muestra, versión de datos) y los mejores hiperparámetros.
- Métricas: MAE por fold como serie con pasos, medias de CV, métricas de test y tiempos.
- Etiquetas de trazabilidad: commit de Git, SHA-256 del dataset y versiones de Python y de las librerías.
- El dataset de entrada (log_input).
- Artefactos: resultados del grid, gráficas, reporte de calidad y requirements.
- El modelo con firma de entrada y salida y un ejemplo de entrada.

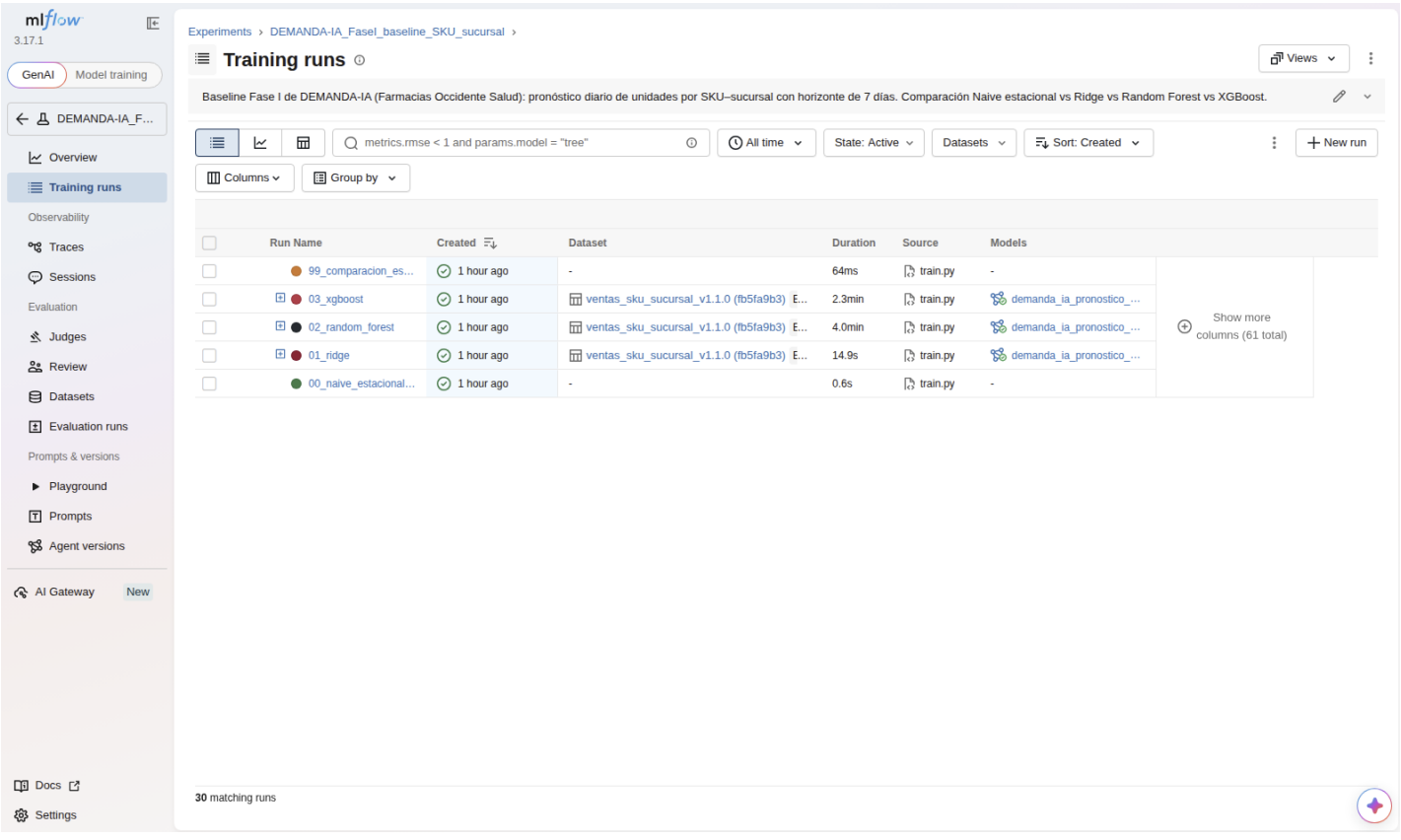


Figura 11. Corridas del experimento, con dataset, duración, código de origen y modelo registrado.

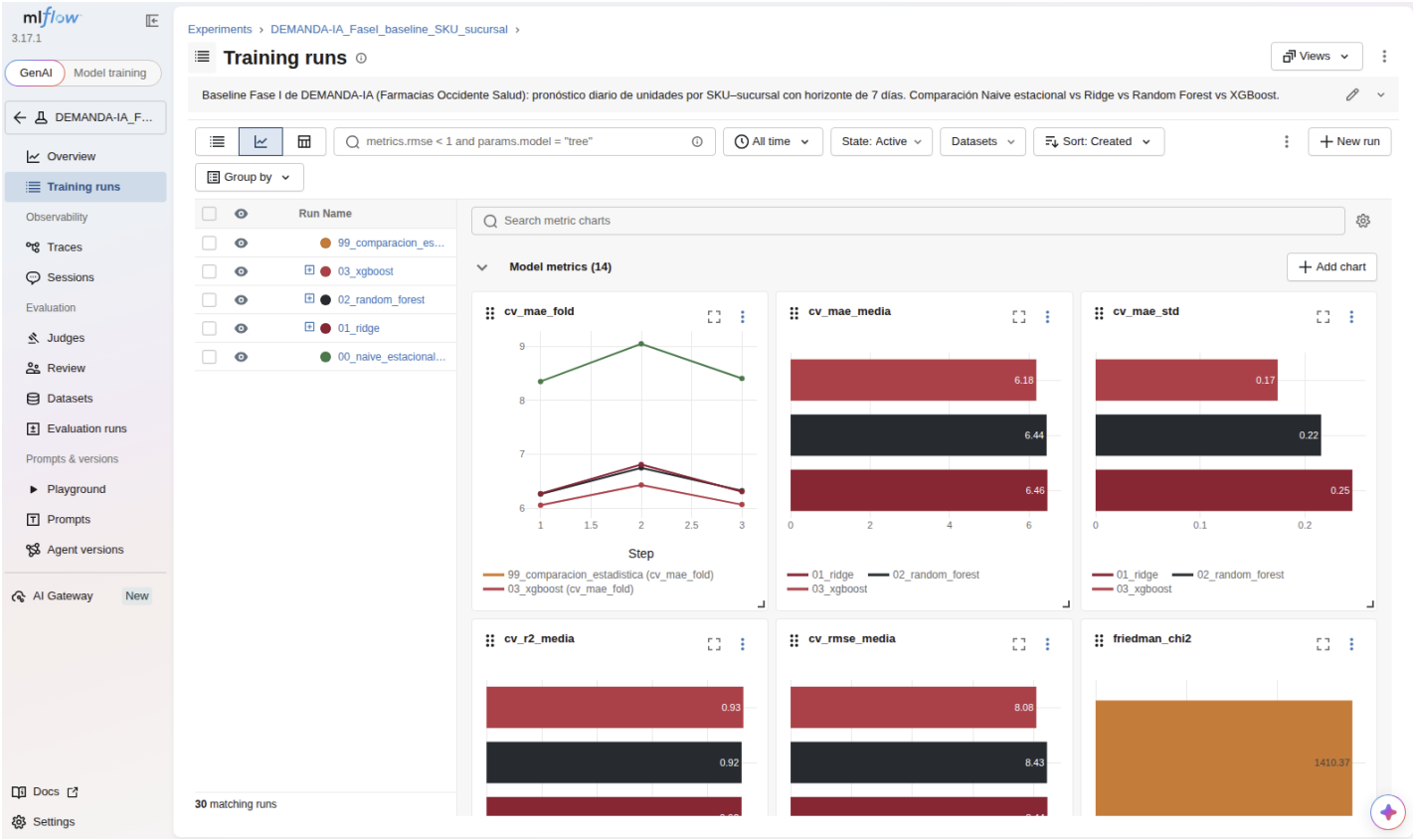


Figura 12. Vista de gráficas: MAE por fold, MAE de test y estadístico de Friedman.

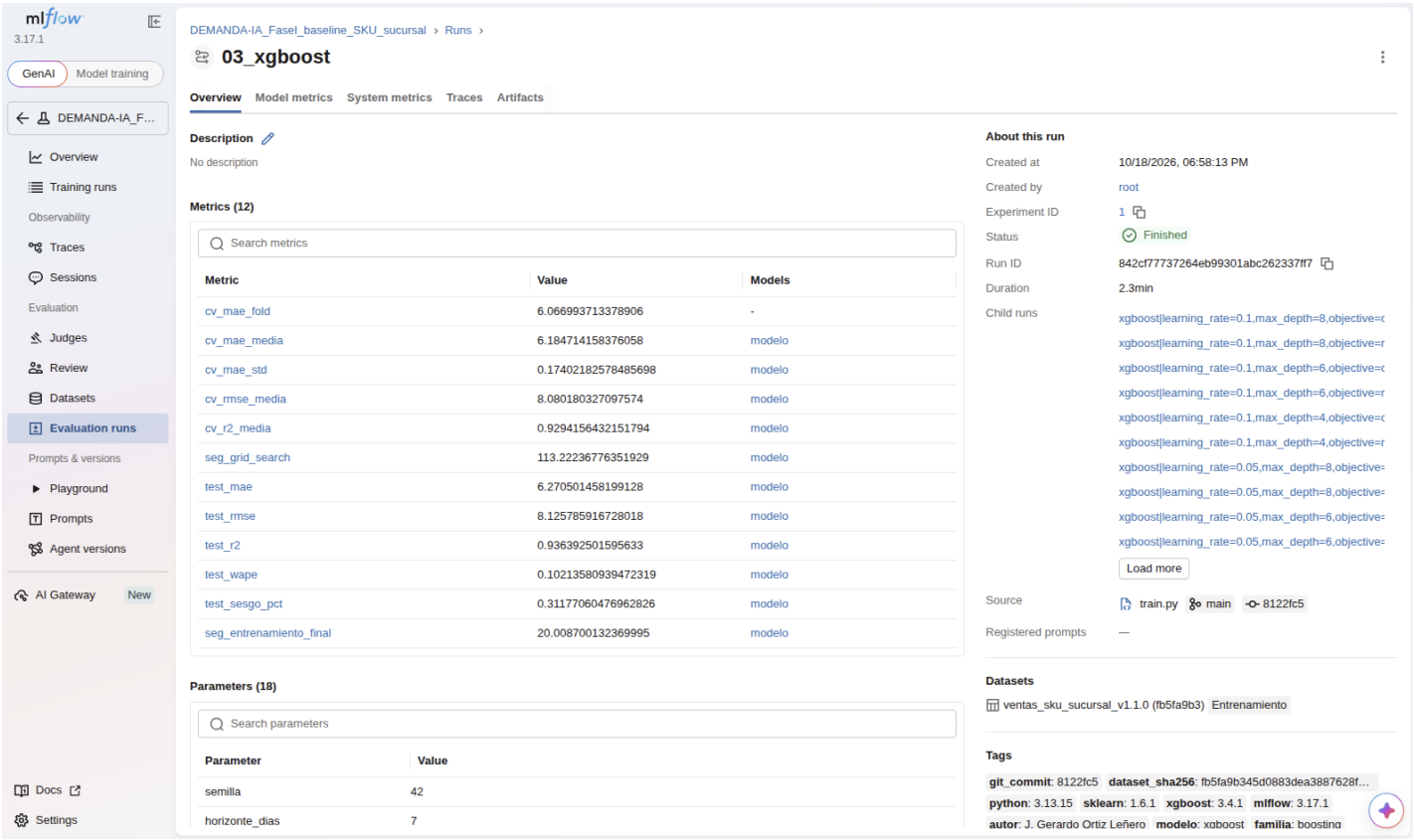


Figura 13. Corrida 03_xgboost: métricas, corridas hijas del grid, dataset, etiquetas de trazabilidad y modelo registrado.

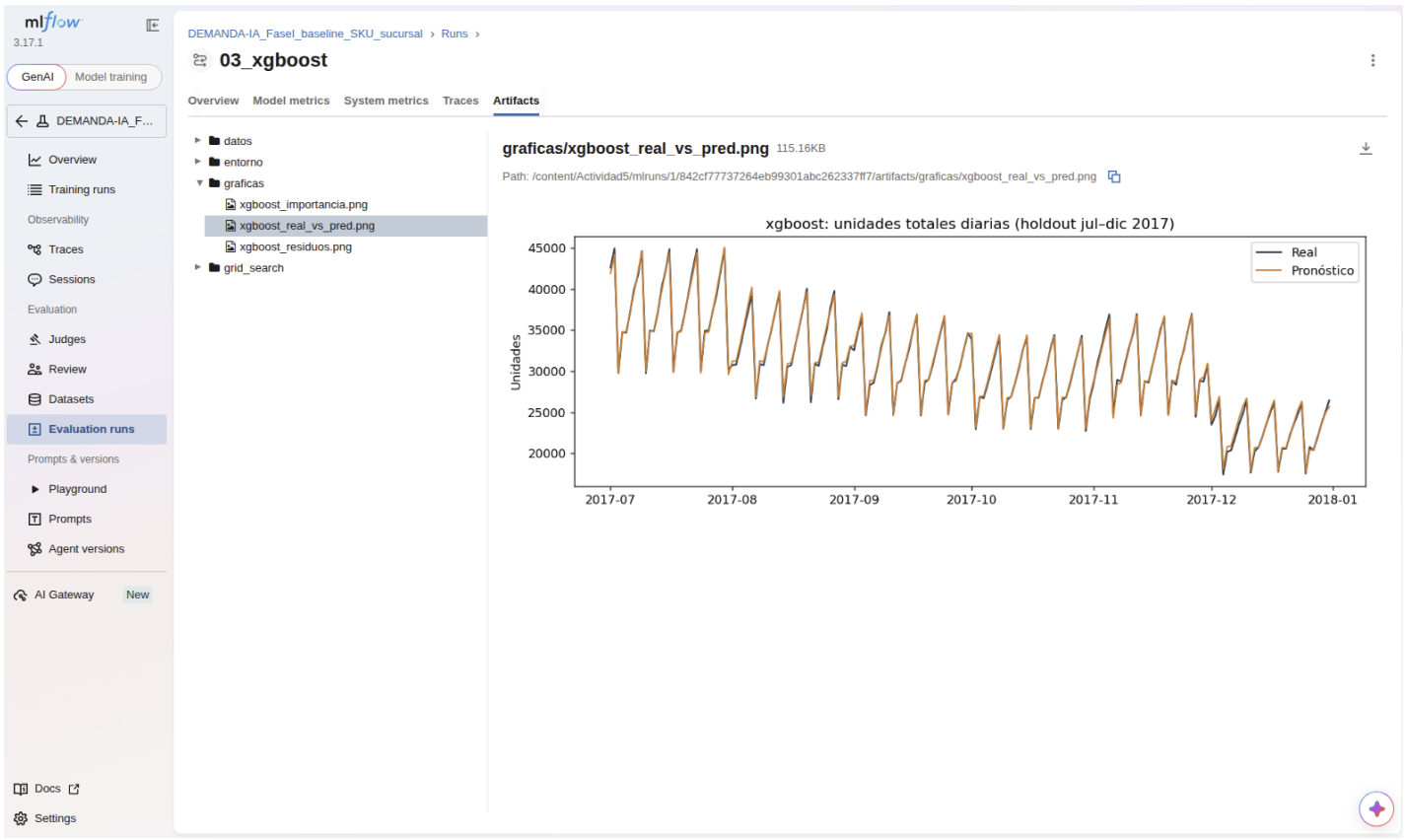


Figura 14. Artefactos de la corrida: carpetas datos, entorno, graficas y grid_search.

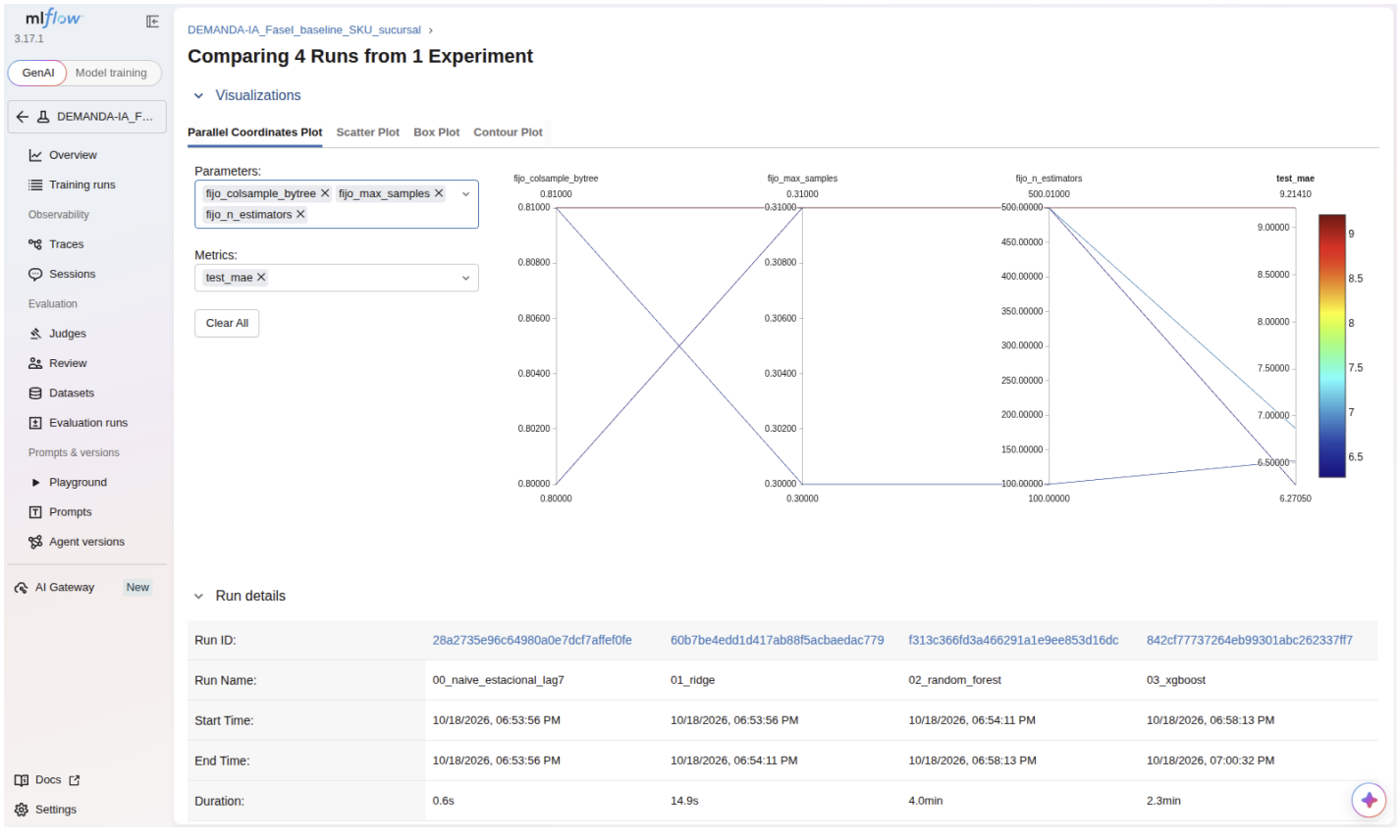


Figura 15. Comparación de las cuatro corridas principales en coordenadas paralelas.

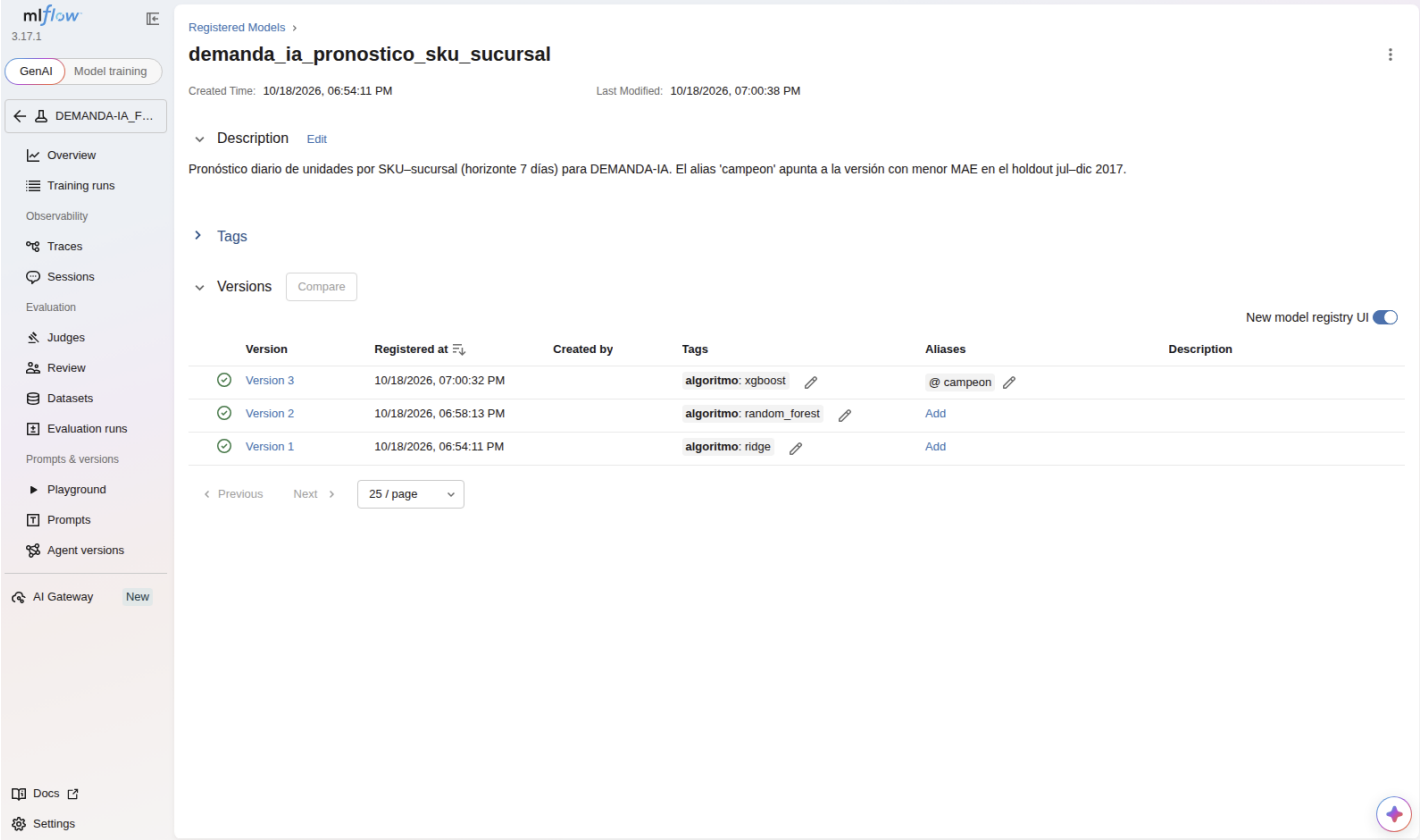


Figura 16. Registro de modelos: versiones 1–3, etiquetadas por algoritmo, con el alias «campeon» en la versión 3.

El proceso nocturno no carga una versión fija del modelo, sino la que tenga asignado el alias campeon en el registro de MLflow (models:/demanda_ia_pronostico_sku_sucursal@campeon). Así, para poner en producción un modelo nuevo basta con mover el alias a la nueva versión, sin tocar el código. La sección 7 de entrena.ipynb comprueba que el modelo se carga correctamente desde el registro usando ese alias.

10.- Deuda técnica y MLOps

Sculley et al. (2015) advierten que el código del modelo es una fracción mínima de un sistema de ML en producción. La deuda técnica se acumula en los datos, en las dependencias y en la infraestructura que lo rodea. La Tabla 9 resume la deuda identificada en este baseline, con la práctica MLOps que la resuelve y su prioridad para la Fase I.

Deuda identificada	Evidencia en esta actividad	Acción MLOps propuesta	Prioridad
Datos proxy demasiado regulares	Correlación 0.9999 entre sucursales; CV de 0.26–0.37; sin intermitencia	Sustituir por el extracto del ERP (datos-v2.0.0); repetir el EDA y el grid. Esperar mayor WAPE y reevaluar la pérdida Poisson/Tweedie	Alta
Demanda censurada	Ventas = demanda solo si hay inventario	Incorporar existencias; excluir o corregir días con quiebre de stock antes de entrenar	Alta
Riesgo de sesgo entrenamiento-servicio	Las variables se calculan en pandas dentro de datos_prep.py	Empaquetar la ingeniería de variables como paso único reutilizado en	Alta

		entrenamiento e inferencia (pipeline o feature store); pruebas de equivalencia	
Artefactos binarios grandes	Random Forest de más de 100 MB excluido de Git	Almacén de artefactos remoto (S3/MinIO) para MLflow; DVC o Git LFS para datos; criterio de tamaño y latencia al elegir modelos	Media
Seguimiento local	mlflow.db en SQLite, un solo usuario	Servidor de MLflow compartido (PostgreSQL + almacén de objetos) con control de acceso	Media
Promoción manual	El alias campeon se asigna por código al mínimo MAE	Puerta de calidad en CI/CD: promover solo si supera al campeón vigente con DM $p < 0.05$ y sesgo < 2 %	Media
Sin monitoreo	No hay registro de deriva ni de error en producción	Monitorear WAPE y sesgo por sucursal cada semana, deriva de distribuciones (PSI) y reentrenamiento programado o por umbral	Alta
Grid sobre muestra y rejilla pequeña	12 % de los datos; 25 combinaciones	Búsqueda bayesiana (p. ej., Optuna) registrada en MLflow, con presupuesto de cómputo definido	Baja
Pronóstico puntual	Solo la media; el reabasto necesita incertidumbre	Pronóstico por cuantiles (p50/p90) para calcular el stock de seguridad por nivel de servicio	Media
Calendario y exógenas ausentes	Sin quincenas, festivos mexicanos ni temporada respiratoria	Tabla de calendario versionada y variables epidemiológicas como fuentes de datos con contrato	Media
Pruebas incompletas	Solo la limpieza tiene prueba automatizada	Pruebas de esquema (p. ej., Great Expectations), de variables y de humo del entrenamiento en CI	Media
Ejecución desde cuaderno	El cuaderno invoca train.py	Orquestar el lote nocturno (Airflow, Prefect o un CronJob en OpenShift) con train.py como unidad; el cuaderno queda solo para análisis	Media

Tabla 9. Registro de deuda técnica y acciones bajo enfoque MLOps.

Decisión con evidencia. Se recomienda llevar XGBoost (versión 3 del registro) como baseline de la Fase I por cuatro razones:

- Tiene el menor error en el holdout y en la validación cruzada, con superioridad estadísticamente significativa sobre todas las alternativas.
- Es el más estable entre folds (d.e. de 0.17).
- Entrena en 20.0 s frente a 71.9 s del Random Forest.
- Su artefacto pesa 2.5 MB frente a más de 100 MB, una ventaja operativa directa para un lote nocturno sobre 140 sucursales.

Ridge debe conservarse como modelo de respaldo interpretable y como control de cordura en el monitoreo. La métrica comprometida con el negocio debe ser el WAPE por sucursal, junto con el sesgo, y no el R^2: el R^2 es alto incluso para la referencia naive (0.86) porque la estacionalidad explica gran parte de la varianza.

11.- Conclusión

Se construyó el primer baseline reproducible de DEMANDA-IA. El dataset SKU–sucursal se limpió con reglas explícitas y probadas, y quedó versionado con Git y huellas SHA-256. Tres algoritmos se ajustaron con Grid Search y validación cruzada temporal libre de fuga de información, y XGBoost resultó campeón con un WAPE de 10.2 %, 32 % mejor que la referencia.

La superioridad se sustentó con pruebas no paramétricas, paramétricas y de Diebold–Mariano.

MLflow concentra la trazabilidad completa: código, datos, parámetros, métricas, artefactos y modelos registrados con un alias de producción. El paso crítico siguiente no es mejorar el algoritmo, sino sustituir el proxy por los datos reales con inventario e instrumentar el monitoreo, porque ahí está la deuda técnica de mayor riesgo para la Fase I.

Declaración de uso de herramientas de inteligencia artificial

Este trabajo se apoyó en Claude (Anthropic) para escribir el código de limpieza, entrenamiento y el notebook. El dataset es público (Kaggle, 2018). La ejecución del entrenamiento, el registro en MLflow y la toma de evidencias en Google Colab, la publicación y organización del repositorio en GitHub, la revisión de resultados y la edición final del documento son de mi autoría.

Bibliografía

Bergmeir, C., & Benítez, J. M. (2012). *On the use of cross-validation for time series predictor evaluation. Information Sciences, 191, 192–213.* Obtenido de https://doi.org/10.1016/j.ins.2011.12.028

Breiman, L. (2001). *Random forests. Machine Learning, 45(1), 5–32.* Obtenido de https://doi.org/10.1023/A:1010933404324

Chen, T., & Guestrin, C. . (2016). *XGBoost: A scalable tree boosting system. En Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794)*. Obtenido de https://doi.org/10.1145/2939672.2939785

Demšar, J. . (2006). *Statistical comparisons of classifiers over multiple data sets. Journal of Machine Learning Research, 7, 1–30.*

Diebold, F. X., & Mariano, R. S. (1995). *Comparing predictive accuracy. Journal of Business & Economic Statistics, 13(3), 253–263.*

Harvey, D., Leybourne, S., & Newbold, P. . (1997). *Testing the equality of prediction mean squared errors. International Journal of Forecasting, 13(2), 281–291.*

Hoerl, A. E., & Kennard, R. W. (1970). *Ridge regression: Biased estimation for nonorthogonal problems. Technometrics, 12(1), 55–67.*

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice (3.ª ed.). OTexts.* Obtenido de https://otexts.com/fpp3/

Iglewicz, B., & Hoaglin, D. C. (1993). *How to detect and handle outliers. ASQC Quality Press.*

Kaggle. (2018). Obtenido de Store Item Demand Forecasting Challenge [Conjunto de datos]: https://www.kaggle.com/competitions/demand-forecasting-kernels-only

MLflow Project. (2026). *MLflow documentation: Tracking and Model Registry.* Obtenido de https://mlflow.org/docs/latest/

Pedregosa, F., et al. (2011). *Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.*

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). *Hidden technical debt in machine learning systems. Advances in Neural Information Processing Systems, 28, 2503–2511.*

Universidad Tecmilenio. (s.f.). *Entrenamiento y ajuste de modelos [Material del curso Gestión de proyectos de inteligencia artificial]*. Obtenido de https://cursos.tecmilenio.mx/courses/252869

Universidad Tecmilenio. (s.f.). *Tema 11. Gestión de datos para entrenamiento [Material del curso Gestión de proyectos de inteligencia artificial]*. Obtenido de https://cursos.tecmilenio.mx/courses/252869